Using AI to improve your code

Eris :3 (they/them)

October 23, 2026

I FUCKING HATE AI!!!

I FUCKING HATE AI!!!

Please imagine a video of Sam Altman being blown up by a missile.

I FUCKING HATE AI!!!

Please imagine a video of Sam Altman being blown up by a missile.
These are my thoughts on ai :)

I FUCKING HATE AI!!!

Please imagine a video of Sam Altman being blown up by a missile.
These are my thoughts on ai :) Artificial intelligence will not be mentioned again for the rest of this talk.

Using ~~AI~~ Abstract Interpretation to improve your code

Eris :3 (they/them)

October 23, 2026

How other people use Abstract Interpretation to improve your code when you run it

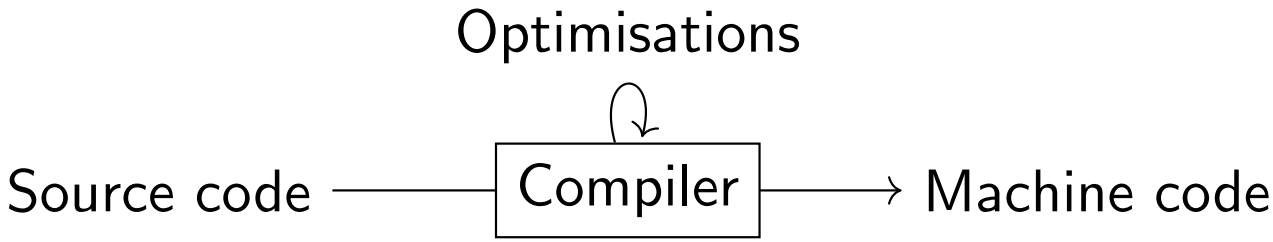
Eris :3 (they/them)

October 23, 2026

Optimising compilers



Optimising compilers



Fibonacci program

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
if $\neg(0 \leq 0 \leq 99 \wedge 0 \leq 1 \leq 99)$ **then**
    explode (out of bounds)
end if
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    if $\neg(0 \leq i - 2 \leq 99 \wedge 0 \leq i - 1 \leq 99 \wedge 0 \leq i \leq 99)$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i - 1] + F[i - 2]$
    $i \leftarrow i + 1$
end while
return F

Fibonacci program

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
if false **then**
    explode (out of bounds)
end if
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    if $\neg(0 \leq i-2 \leq 99 \wedge 0 \leq i-1 \leq 99 \wedge 0 \leq i \leq 99)$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$
    $i \leftarrow i+1$
end while
return F

Fibonacci program

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    if $\neg(0 \leq i - 2 \leq 99 \wedge 0 \leq i - 1 \leq 99 \wedge 0 \leq i \leq 99)$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i - 1] + F[i - 2]$
    $i \leftarrow i + 1$
end while
return F

Fibonacci program

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    if $i < 2 \vee 99 < i$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$
    $i \leftarrow i + 1$
end while
return F

Fibonacci program

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    if $i < 2 \vee 99 < i$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$
    $i \leftarrow i + 1$
end while
return F

The bounds check is redundant though!

Syntax

We define expressions, conditions and statements.

$$
\begin{array}{rcl}
s & := & v \leftarrow e \mid s; s \mid \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \\
& & \mid \textbf{while } c \textbf{ do } s \textbf{ end while} \mid \textbf{explode} \mid \textbf{skip}
\end{array}
$$

Syntax

We define expressions, conditions and statements.
Expressions are things that evaluate to numbers.

$$
\begin{aligned}
s \quad := \quad & v \leftarrow e \mid s; s \mid \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \\
& \mid \textbf{while } c \textbf{ do } s \textbf{ end while} \mid \textbf{explode} \mid \textbf{skip}
\end{aligned}
$$

Syntax

We define expressions, conditions and statements.
Expressions are things that evaluate to numbers.

- A number n is an expression.

$$s \quad := \quad v \leftarrow e \;\mid\; s; s \;\mid\; \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \\ \mid\; \textbf{while } c \textbf{ do } s \textbf{ end while} \;\mid\; \textbf{explode} \;\mid\; \textbf{skip}$$

Syntax

We define expressions, conditions and statements.
Expressions are things that evaluate to numbers.

- A number n is an expression.
- A variable (integer variable) v is an expression.

$$s \quad := \quad v \leftarrow e \;\mid\; s; s \;\mid\; \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \\ \mid\; \textbf{while } c \textbf{ do } s \textbf{ end while} \;\mid\; \textbf{explode} \;\mid\; \textbf{skip}$$

Syntax

We define expressions, conditions and statements.
Expressions are things that evaluate to numbers.

- A number n is an expression.
- A variable (integer variable) v is an expression.
- Index an array by an expression $a[e]$.

$$s \quad := \quad v \leftarrow e \mid s; s \mid \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \mid \textbf{while } c \textbf{ do } s \textbf{ end while} \mid \textbf{explode} \mid \textbf{skip}$$

Syntax

We define expressions, conditions and statements.
Expressions are things that evaluate to numbers.

- A number n is an expression.
- A variable (integer variable) v is an expression.
- Index an array by an expression $a[e]$.
- We can write down the sum of two expressions $e_1 + e_2$.

$$
\begin{aligned}
s \quad := \quad & v \leftarrow e \mid s; s \mid \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \\
& \mid \textbf{while } c \textbf{ do } s \textbf{ end while} \mid \textbf{explode} \mid \textbf{skip}
\end{aligned}
$$

Syntax

We define expressions, conditions and statements.
Expressions are things that evaluate to numbers.

- A number n is an expression.
- A variable (integer variable) v is an expression.
- Index an array by an expression $a[e]$.
- We can write down the sum of two expressions $e_1 + e_2$.
- Difference $e_1 - e_2$.

$$s \quad := \quad v \leftarrow e \mid s; s \mid \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \mid \textbf{while } c \textbf{ do } s \textbf{ end while} \mid \textbf{explode} \mid \textbf{skip}$$

Syntax

We define expressions, conditions and statements.
Expressions are things that evaluate to numbers.

- A number n is an expression.
- A variable (integer variable) v is an expression.
- Index an array by an expression $a[e]$.
- We can write down the sum of two expressions $e_1 + e_2$.
- Difference $e_1 - e_2$.
- Product $e_1 \times e_2$.

$$s \quad := \quad v \leftarrow e \mid s; s \mid \textbf{if } c \textbf{ then } s \textbf{ else } s \textbf{ end if} \mid \textbf{while } c \textbf{ do } s \textbf{ end while} \mid \textbf{explode} \mid \textbf{skip}$$

Syntax

Conditions are what guard ifs and whiles.

Syntax

Conditions are what guard ifs and whiles.

- true and false are conditions

Syntax

Conditions are what guard ifs and whiles.

- true and false are conditions
- Comparisons of expressions like $e_1 < e_2$ or $e_1 = e_2$.

Syntax

Conditions are what guard ifs and whiles.

- true and false are conditions
- Comparisons of expressions like $e_1 < e_2$ or $e_1 = e_2$.
- Conjunctions $c_1 \wedge c_2$

Syntax

Conditions are what guard ifs and whiles.

- true and false are conditions
- Comparisons of expressions like $e_1 < e_2$ or $e_1 = e_2$.
- Conjunctions $c_1 \wedge c_2$
- Disjunctions $c_1 \vee c_2$

Syntax

Conditions are what guard ifs and whiles.

- true and false are conditions
- Comparisons of expressions like $e_1 < e_2$ or $e_1 = e_2$.
- Conjunctions $c_1 \wedge c_2$
- Disjunctions $c_1 \vee c_2$

Negation is dealt with by applying de-morgan's law and negating "atoms".

$$\neg(x \leq y \wedge y \leq z) = x > y \vee y > z$$

Syntax

Statements are the lines of code.

Syntax

Statements are the lines of code.

- Variable assignment by expressions $v \leftarrow e$

Syntax

Statements are the lines of code.

- Variable assignment by expressions $v \leftarrow e$
- Array updates, indexed by expressions $a[e_1] \leftarrow e_2$

Syntax

Statements are the lines of code.

- Variable assignment by expressions $v \leftarrow e$
- Array updates, indexed by expressions $a[e_1] \leftarrow e_2$
- The composite of two statements $s_1; s_2$

Syntax

Statements are the lines of code.

- Variable assignment by expressions $v \leftarrow e$
- Array updates, indexed by expressions $a[e_1] \leftarrow e_2$
- The composite of two statements $s_1; s_2$
- If statements, with bodies statements **if** c **then** s_1 **else** s_2 **end if**

Syntax

Statements are the lines of code.

- Variable assignment by expressions $v \leftarrow e$
- Array updates, indexed by expressions $a[e_1] \leftarrow e_2$
- The composite of two statements $s_1; s_2$
- If statements, with bodies statements **if** c **then** s_1 **else** s_2 **end if**
- While loops, with bodies statements **while** c **do** s **end while**

Semantics

There is a set $\mathcal{E}$ of states (assignments of all variables to values) and (letting $\mathcal{D} = \mathcal{P}(\mathcal{E})$) an operator $\mathbb{S}[\![s]\!] : \mathcal{D} \to \mathcal{D}$ for each statement s. This operator describes "what the statements do".

Semantics

There is a set $\mathcal{E}$ of states (assignments of all variables to values) and (letting $\mathcal{D} = \mathcal{P}(\mathcal{E})$) an operator $\mathbb{S}[\![s]\!] : \mathcal{D} \to \mathcal{D}$ for each statement s. This operator describes "what the statements do". For example,

$$\mathbb{S}[\![x \leftarrow 5]\!](\mathcal{E}) = \{\sigma \in \mathcal{E} \mid \sigma(x) = 5\}.$$

Semantics

There is a set $\mathcal{E}$ of states (assignments of all variables to values) and (letting $\mathcal{D} = \mathcal{P}(\mathcal{E})$) an operator $\mathbb{S}[\![s]\!] : \mathcal{D} \to \mathcal{D}$ for each statement s. This operator describes "what the statements do". For example,

$$\mathbb{S}[\![x \leftarrow 5]\!](\mathcal{E}) = \{\sigma \in \mathcal{E} \mid \sigma(x) = 5\}.$$

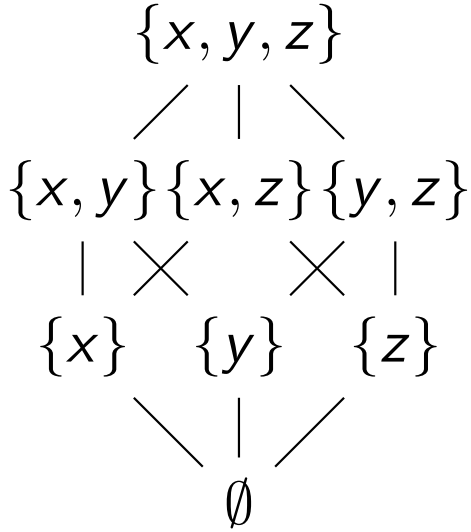
It's quite yucky to write out everything in full with arrays included. Instead, we will use order theory to "approximate" the real semantics.

Order theory 1

For any set S, $(\mathcal{P}(S), \subseteq)$ is a poset.

Order theory 1

For any set S, $(\mathcal{P}(S), \subseteq)$ is a poset.



Order theory 1

Definition

A map $f : P \to Q$ between posets is monotone if whenever $x \sqsubseteq_P y$, $f(x) \sqsubseteq_Q f(y)$.

Order theory 1

Definition

A map $f : P \to Q$ between posets is monotone if whenever $x \sqsubseteq_P y$, $f(x) \sqsubseteq_Q f(y)$.

Proposition

The identity map is monotone. Composites of monotone maps are monotone.

Approximating semantics

Instead of defining the full semantics, define an approximation set of states $\hat{\mathcal{E}}$ and (letting $\hat{\mathcal{D}} = \mathcal{P}(\hat{\mathcal{E}})$) define an approximation operator $\hat{\mathbb{S}}[\![s]\!] : \hat{\mathcal{D}} \to \hat{\mathcal{D}}$ for each statement s.

Approximating semantics

Instead of defining the full semantics, define an approximation set of states $\hat{\mathcal{E}}$ and (letting $\hat{\mathcal{D}} = \mathcal{P}(\hat{\mathcal{E}})$) define an approximation operator $\hat{\mathbb{S}}[\![s]\!] : \hat{\mathcal{D}} \to \hat{\mathcal{D}}$ for each statement s.
Need a monotone map $\hat{\gamma} : \hat{\mathcal{D}} \to \mathcal{D}$ such that for any set of states S, $\mathbb{S}[\![s]\!](\gamma(S)) \subseteq \gamma(\hat{\mathbb{S}}[\![s]\!](S))$. Intuitively, every state described in the "real world" is captured in the "approximate world".

Approximating semantics

Instead of defining the full semantics, define an approximation set of states $\hat{\mathcal{E}}$ and (letting $\hat{\mathcal{D}} = \mathcal{P}(\hat{\mathcal{E}})$) define an approximation operator $\hat{\mathbb{S}}[\![s]\!] : \hat{\mathcal{D}} \to \hat{\mathcal{D}}$ for each statement s.
Need a monotone map $\hat{\gamma} : \hat{\mathcal{D}} \to \mathcal{D}$ such that for any set of states S, $\mathbb{S}[\![s]\!](\gamma(S)) \subseteq \gamma(\hat{\mathbb{S}}[\![s]\!](S))$. Intuitively, every state described in the "real world" is captured in the "approximate world".
If you prefer, we need $\hat{\gamma}$ a lax natural transformation viewing semantics opeators as functors from the one object category of statements under sequential composition to the poset-enriched category of posets!

$$\begin{array}{ccc} \hat{\mathcal{D}} & \xrightarrow{\hat{\mathbb{S}}[\![s]\!]} & \hat{\mathcal{D}} \\ {\scriptstyle\gamma}\downarrow & \swarrow & \downarrow{\scriptstyle\gamma} \\ \mathcal{D} & \xrightarrow[\mathbb{S}[\![s]\!]]{} & \mathcal{D} \end{array}$$

Approximating semantics

Define $\hat{\mathcal{E}} = \mathbb{Z}^V$ (where V is the set of *integer* variables only) and $\hat{\mathcal{D}} = \mathcal{P}(\hat{\mathcal{E}})$. Then define $\gamma : \hat{\mathcal{D}} \to \mathcal{D}$, to give all possible assignments of arrays.
Then we'll define $\mathbb{E}[\![e]\!] : \hat{\mathcal{E}} \to \mathcal{P}(\mathbb{Z})$, $\mathbb{C}[\![c]\!] : \hat{\mathcal{D}} \to \hat{\mathcal{D}}$ and $\hat{\mathbb{S}}[\![s]\!] : \hat{\mathcal{D}} \to \hat{\mathcal{D}}$ inductively.
$\mathbb{E}[\![e]\!]$ tells us what expressions can evaluate to and $\mathbb{C}[\![c]\!]$ tells us which statements pass through a guard.

Approximating semantics

$$
\begin{aligned}
\mathbb{E}[\![n]\!](\sigma) &= \{n\} \\
\mathbb{E}[\![v]\!](\sigma) &= \{\sigma(v)\} \\
\mathbb{E}[\![e_1 + e_2]\!](\sigma) &= \{x + y \mid x \in \mathbb{E}[\![e_1]\!](\sigma), y \in \mathbb{E}[\![e_2]\!](\sigma)\} \\
\mathbb{E}[\![e_1 - e_2]\!](\sigma) &= \{x - y \mid x \in \mathbb{E}[\![e_1]\!](\sigma), y \in \mathbb{E}[\![e_2]\!](\sigma)\} \\
\mathbb{E}[\![e_1 \times e_2]\!](\sigma) &= \{x \times y \mid x \in \mathbb{E}[\![e_1]\!](\sigma), y \in \mathbb{E}[\![e_2]\!](\sigma)\} \\
\mathbb{E}[\![a[i]]\!](\sigma) &= \mathbb{Z}
\end{aligned}
$$

$$
\begin{aligned}
\mathbb{C}[\![\text{true}]\!](S) &= S \\
\mathbb{C}[\![\text{false}]\!](S) &= \emptyset \\
\mathbb{C}[\![e_1 \leq e_2]\!](S) &= \{\sigma \in S \mid \exists x \in \mathbb{E}[\![e_1]\!], y \in \mathbb{E}[\![e_2]\!] : x \leq y\} \\
\mathbb{C}[\![e_1 = e_2]\!](S) &= \{\sigma \in S \mid \exists x \in \mathbb{E}[\![e_1]\!], y \in \mathbb{E}[\![e_2]\!] : x = y\} \\
\mathbb{C}[\![c_1 \wedge c_2]\!](S) &= \mathbb{C}[\![c_1]\!](S) \cap \mathbb{C}[\![c_2]\!](S) \\
\mathbb{C}[\![c_1 \vee c_2]\!](S) &= \mathbb{C}[\![c_1]\!](S) \cup \mathbb{C}[\![c_2]\!](S)
\end{aligned}
$$

Approximating semantics

$$\hat{\mathbb{S}}[\![s_1; s_2]\!](S) = \hat{\mathbb{S}}[\![s_2]\!](\hat{\mathbb{S}}[\![s_1]\!](S))$$
$$\hat{\mathbb{S}}[\![v \leftarrow e]\!](S) = \{\sigma[v \mapsto x] \mid \sigma \in S, x \in \mathbb{E}[\![e]\!](\sigma)\}$$
$$\hat{\mathbb{S}}[\![a[i] \leftarrow e]\!](S) = S$$
$$\hat{\mathbb{S}}[\![\textbf{if } c \textbf{ then } s_1 \textbf{ else } s_2 \textbf{ end if}]\!](S) = \hat{\mathbb{S}}[\![s_1]\!](\mathbb{C}[\![c]\!](S)) \cup \hat{\mathbb{S}}[\![s_1]\!](\mathbb{C}[\![\neg c]\!](S))$$

Approximating semantics

$$\hat{\mathbb{S}}[\![s_1; s_2]\!](S) = \hat{\mathbb{S}}[\![s_2]\!](\hat{\mathbb{S}}[\![s_1]\!](S))$$
$$\hat{\mathbb{S}}[\![v \leftarrow e]\!](S) = \{\sigma[v \mapsto x] \mid \sigma \in S, x \in \mathbb{E}[\![e]\!](\sigma)\}$$
$$\hat{\mathbb{S}}[\![a[i] \leftarrow e]\!](S) = S$$
$$\hat{\mathbb{S}}[\![\textbf{if } c \textbf{ then } s_1 \textbf{ else } s_2 \textbf{ end if}]\!](S) = \hat{\mathbb{S}}[\![s_1]\!](\mathbb{C}[\![c]\!](S)) \cup \hat{\mathbb{S}}[\![s_1]\!](\mathbb{C}[\![\neg c]\!](S))$$
$$\hat{\mathbb{S}}[\![\textbf{while } c \textbf{ do } s \textbf{ end while}]\!](S) = \text{wait what}$$

Approximating semantics

$$\hat{\mathbb{S}}[\![s_1; s_2]\!](S) = \hat{\mathbb{S}}[\![s_2]\!](\hat{\mathbb{S}}[\![s_1]\!](S))$$
$$\hat{\mathbb{S}}[\![v \leftarrow e]\!](S) = \{\sigma[v \mapsto x] \mid \sigma \in S, x \in \mathbb{E}[\![e]\!](\sigma)\}$$
$$\hat{\mathbb{S}}[\![a[i] \leftarrow e]\!](S) = S$$
$$\hat{\mathbb{S}}[\![\textbf{if } c \textbf{ then } s_1 \textbf{ else } s_2 \textbf{ end if}]\!](S) = \hat{\mathbb{S}}[\![s_1]\!](\mathbb{C}[\![c]\!](S)) \cup \hat{\mathbb{S}}[\![s_1]\!](\mathbb{C}[\![\neg c]\!](S))$$
$$\hat{\mathbb{S}}[\![\textbf{while } c \textbf{ do } s \textbf{ end while}]\!](S) = \text{wait what}$$

We need some sort of limit of repeatedly applying s! Alternatively, we could find some minimal X such that $X = X \cup \hat{\mathbb{S}}[\![s]\!](\mathbb{C}[\![c]\!](X))$.

Order theory 2

Unfortunately domain theorists use ω to mean $\mathbb{N}$.

Order theory 2

Unfortunately domain theorists use ω to mean $\mathbb{N}$.

Definition

An ω-chain is a sequence $(x_i)_{i \in \omega}$ such that if $i \sqsubseteq j$ then $x_i \sqsubseteq x_j$.

Order theory 2

Unfortunately domain theorists use ω to mean $\mathbb{N}$.

Definition

An ω-chain is a sequence $(x_i)_{i \in \omega}$ such that if $i \sqsubseteq j$ then $x_i \sqsubseteq x_j$.

Definition

An ω complete partial order, or ω-CPO is a poset $(C, \sqsubseteq)$ such that all ω-chains (x_i) admit a least upper bound $\bigsqcup_i x_i$ (so $x_j \sqsubseteq \bigsqcup_i x_i$ for all j and is minimal among such). We may call these least upper bounds *joins*. An ω-CPO is *pointed* if it has a least element $\bot$.

Order theory 2

Example

For any set X, the poset $(\mathcal{P}(X), \subseteq)$ is an ω-cpo! Joins are given by unions of sets, $\bigsqcup_i X_i = \bigcup_i X_i$. Moreover, it is pointed by $\emptyset$!

We can think of joins as generalisations of unions, but we will only see instances of non-union joins later.

Order theory 2

Definition

A monotone map $f : C \to D$ of ω-cpos is ω-continuous if for any ω-chain $(x_i)_{i \in \omega}$ in C, $f(\bigsqcup_i x_i) = \bigsqcup_i f(x_i)$.

Theorem (Kleene fixed-point theorem for ω-cpos)

If C is a pointed ω-cpo and $f : C \to C$ is an ω-continuous map, then a fixed point of f exists, and the least fixed point is

$$\mathrm{lfp}(f) = \bigsqcup_i f^i(\bot).$$

Order theory 2

Proof.

Notice that f^i is monotone since it is a composite of monotone maps. Applied to $\bot \sqsubseteq f(\bot)$, we get $f^i(\bot) \sqsubseteq f^{i+1}(\bot)$, so $f^i(\bot)$ does form an ω-chain!



Order theory 2

Proof.

Notice that f^i is monotone since it is a composite of monotone maps. Applied to $\bot \sqsubseteq f(\bot)$, we get $f^i(\bot) \sqsubseteq f^{i+1}(\bot)$, so $f^i(\bot)$ does form an ω-chain!

Next, our computation does give a fixed point, since

$$f\left(\bigsqcup_i f^i(\bot)\right) = \bigsqcup_i f^{i+1}(\bot) = \bigsqcup_i f^i(\bot).$$

□

Order theory 2

Proof.

Notice that f^i is monotone since it is a composite of monotone maps. Applied to $\bot \sqsubseteq f(\bot)$, we get $f^i(\bot) \sqsubseteq f^{i+1}(\bot)$, so $f^i(\bot)$ does form an ω-chain!

Next, our computation does give a fixed point, since

$$f\left(\bigsqcup_i f^i(\bot)\right) = \bigsqcup_i f^{i+1}(\bot) = \bigsqcup_i f^i(\bot).$$

Finally, if x is another fixed-point of f, then applying monotonicity to $\bot \sqsubseteq x$ we get $f^i(\bot) \sqsubseteq f^i(x) = x$, so $\sqsubseteq_i f^i(\bot) \sqsubseteq x$. $\square$

While loops

And now we can define while loop semantics!

While loops

And now we can define while loop semantics!

$$\hat{\mathbb{S}}[\![\textbf{while } c \textbf{ do } s \textbf{ end while}]\!](S) = \mathbb{C}[\![\neg c]\!](\mathsf{lfp}(X \mapsto X \cup \hat{\mathbb{S}}[\![s]\!](\mathbb{C}[\![c]\!](X))))$$

We've finally finished defining the language ...

While loops

And now we can define while loop semantics!

$$\hat{\mathbb{S}}[\![\textbf{while } c \textbf{ do } s \textbf{ end while}]\!](S) = \mathbb{C}[\![\neg c]\!](\mathsf{lfp}(X \mapsto X \cup \hat{\mathbb{S}}[\![s]\!](\mathbb{C}[\![c]\!](X))))$$

We've finally finished defining the language ...
(There are questions about how we can know the inner expression is continuous if we haven't fully defined it but shhh this talk isn't about induction)

Abstract interpretation

The semantics we just described is very non-computable! Abstract interpretation is about approximating our "concrete" semantics so that it becomes computable.

Abstract interpretation

The semantics we just described is very non-computable! Abstract interpretation is about approximating our "concrete" semantics so that it becomes computable.
That is, we want a poset $(A, \sqsubseteq)$ with elements of A easy to represent on a computer, and some computable semantics operator $\mathbb{S}^\sharp[\![s]\!] : A \to A$ for each statement s, that's an approximation of our semantics by a map $\gamma : A \to \hat{\mathcal{D}}$.

Abstract interpretation

The semantics we just described is very non-computable! Abstract interpretation is about approximating our "concrete" semantics so that it becomes computable.

That is, we want a poset $(A, \sqsubseteq)$ with elements of A easy to represent on a computer, and some computable semantics operator $\mathbb{S}^\sharp[\![s]\!] : A \to A$ for each statement s, that's an approximation of our semantics by a map $\gamma : A \to \hat{\mathcal{D}}$.

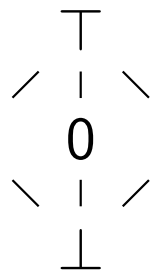
One approach is to store the sets of all values a variable could have ($\mathcal{D}' = \mathcal{P}(\mathbb{Z})^V$). We can then approximate $\mathcal{P}(\mathbb{Z})$ by A, and then get some computable set A^V!

Value domains

Examples of approximations of $\mathcal{P}(\mathbb{Z})$:

Value domains

Examples of approximations of $\mathcal{P}(\mathbb{Z})$:

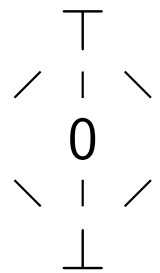
- $S =$ 

.

  $\gamma(+) = \{n \in \mathbb{Z} \mid n > 0\}$ etc.

Value domains

Examples of approximations of $\mathcal{P}(\mathbb{Z})$:

- $S =$ 

  $\gamma(+) = \{n \in \mathbb{Z} \mid n > 0\}$ etc.
- $A = \{[a, b] \mid a, b \in \mathbb{Z} \cup \{-\infty, \infty\}\}$ ordered by subsets.
  $\gamma([a, b]) = \{n \in \mathbb{Z} \mid a \leq n \leq b\}$.

Example again

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$
    $i \leftarrow i + 1$
end while
return F

At the second bounds check, $i \in [2, 99]$!!

Value set semantics

We define just $\mathbb{E}'[\![e]\!] : \mathcal{P}(\mathbb{Z})^V \to \mathcal{P}(\mathbb{Z})$ and leave the rest for later.

Value set semantics

We define just $\mathbb{E}'[\![e]\!] : \mathcal{P}(\mathbb{Z})^V \to \mathcal{P}(\mathbb{Z})$ and leave the rest for later.

$$
\begin{array}{rcl}
\mathbb{E}'[\![n]\!](\sigma) & = & \{n\} \\
\mathbb{E}'[\![v]\!](\sigma) & = & \sigma(v) \\
\mathbb{E}'[\![e_1 + e_2]\!](\sigma) & = & \{x + y \mid x \in \mathbb{E}'[\![e_1]\!](\sigma), y \in \mathbb{E}'[\![e_2]\!](\sigma)\} \\
\mathbb{E}'[\![e_1 - e_2]\!](\sigma) & = & \{x - y \mid x \in \mathbb{E}'[\![e_1]\!](\sigma), y \in \mathbb{E}'[\![e_2]\!](\sigma)\} \\
\mathbb{E}'[\![e_1 \times e_2]\!](\sigma) & = & \{x \times y \mid x \in \mathbb{E}'[\![e_1]\!](\sigma), y \in \mathbb{E}'[\![e_2]\!](\sigma)\} \\
\mathbb{E}'[\![a[i]]\!](\sigma) & = & \mathbb{Z}
\end{array}
$$

Value set semantics

We define just $\mathbb{E}'[\![e]\!] : \mathcal{P}(\mathbb{Z})^V \to \mathcal{P}(\mathbb{Z})$ and leave the rest for later.

$$
\begin{array}{rcl}
\mathbb{E}'[\![n]\!](\sigma) & = & \{n\} \\
\mathbb{E}'[\![v]\!](\sigma) & = & \sigma(v) \\
\mathbb{E}'[\![e_1 + e_2]\!](\sigma) & = & \{x + y \mid x \in \mathbb{E}'[\![e_1]\!](\sigma), y \in \mathbb{E}'[\![e_2]\!](\sigma)\} \\
\mathbb{E}'[\![e_1 - e_2]\!](\sigma) & = & \{x - y \mid x \in \mathbb{E}'[\![e_1]\!](\sigma), y \in \mathbb{E}'[\![e_2]\!](\sigma)\} \\
\mathbb{E}'[\![e_1 \times e_2]\!](\sigma) & = & \{x \times y \mid x \in \mathbb{E}'[\![e_1]\!](\sigma), y \in \mathbb{E}'[\![e_2]\!](\sigma)\} \\
\mathbb{E}'[\![a[i]]\!](\sigma) & = & \mathbb{Z}
\end{array}
$$

We want to approximate $\mathbb{E}'[\![e]\!]$ with intervals instead of sets.

Order theory 3

oops this is out of place but needs to go somewhere

Order theory 3

oops this is out of place but needs to go somewhere

Definition (Non-binary double feature AND lattice double feature?!)

A poset $(P, \sqsubseteq)$ is a join semi-lattice if every pair of elements x, y has a least upper bound (join) written $x \sqcup y$.

Order theory 3

oops this is out of place but needs to go somewhere

Definition (Non-binary double feature AND lattice double feature?!)

A poset $(P, \sqsubseteq)$ is a join semi-lattice if every pair of elements x, y has a least upper bound (join) written $x \sqcup y$.

Example

As usual $(\mathcal{P}(X), \sqsubseteq)$ is a join semi-lattice, where joins are unions!

Order theory 3

Definition

If $(C, \sqsubseteq)$ and $(A, \sqsubseteq)$ are posets, and $\alpha : C \rightleftarrows A : \gamma$ are monotone maps, then they form a Galois connection if $\alpha(c) \sqsubseteq a$ iff $c \sqsubseteq \gamma(a)$. We call α the left adjoint and γ the right adjoint.

Order theory 3

Definition

If $(C, \sqsubseteq)$ and $(A, \sqsubseteq)$ are posets, and $\alpha : C \rightleftarrows A : \gamma$ are monotone maps, then they form a Galois connection if $\alpha(c) \sqsubseteq a$ iff $c \sqsubseteq \gamma(a)$. We call α the left adjoint and γ the right adjoint.

Example

The map $\gamma([a, b]) = \{n \in \mathbb{Z} \mid a \leq n \leq b\}$ has left adjoint $\alpha(S) = [\min(S), \max(S)]$!

Order theory 3

Galois embeddings let us optimally approximate concrete structure in the abstract structure.

Order theory 3

Galois embeddings let us optimally approximate concrete structure in the abstract structure.

Proposition

In a Galois connection $\alpha : C \rightleftarrows A : \gamma$, if C is a join semi-lattice, then A is a join semi-lattice with join $a \sqcup b = \alpha(\gamma(a) \sqcup \gamma(b))$.

Proposition

Any operator $f : C \to C$ has its "best abstraction" on A (won't be defined sorry) by $\alpha \circ f \circ \gamma$.

Order theory 3

Galois embeddings let us optimally approximate concrete structure in the abstract structure.

Proposition

In a Galois connection $\alpha : C \rightleftarrows A : \gamma$, if C is a join semi-lattice, then A is a join semi-lattice with join $a \sqcup b = \alpha(\gamma(a) \sqcup \gamma(b))$.

Proposition

Any operator $f : C \to C$ has its "best abstraction" on A (won't be defined sorry) by $\alpha \circ f \circ \gamma$.

(I'm too dead to write the proofs for this... do them as exercises!)

Order theory 3

Galois embeddings let us optimally approximate concrete structure in the abstract structure.

Proposition

In a Galois connection $\alpha : C \rightleftarrows A : \gamma$, if C is a join semi-lattice, then A is a join semi-lattice with join $a \sqcup b = \alpha(\gamma(a) \sqcup \gamma(b))$.

Proposition

Any operator $f : C \to C$ has its "best abstraction" on A (won't be defined sorry) by $\alpha \circ f \circ \gamma$.

(I'm too dead to write the proofs for this... do them as exercises!) As a general idea, apply γ, then some operation, then α. This gives us approximation operators for free yay!

Computing Interval semantics

Let's write out what $\mathbb{E}^\sharp[\![e_1 + e_2]\!] : I^V \to I$ might be.

Computing Interval semantics

Let's write out what $\mathbb{E}^\sharp[\![e_1 + e_2]\!] : I^V \to I$ might be. Let $[a, b] = \mathbb{E}^\sharp[\![e_1]\!](\sigma^\sharp), [c, d] = \mathbb{E}^\sharp[\![e_2]\!](\sigma^\sharp)$.

$$\begin{aligned}
\mathbb{E}^\sharp[\![e_1 + e_2]\!](\sigma^\sharp) &= \alpha(\mathbb{E}'[\![e_1 + e_2]\!](\gamma(\sigma^\sharp))) \\
&\sqsubseteq \alpha(\{x + y \mid x \in [a, b], y \in [c, d]\}) \\
&= [\min(\{x + y \mid x \in [a, b], y \in [c, d]\}), \max(\cdots)] \\
&= [a + c, b + d]
\end{aligned}$$

Computing Interval semantics

Let's write out what $\mathbb{E}^\sharp[\![e_1 + e_2]\!] : I^V \to I$ might be. Let $[a, b] = \mathbb{E}^\sharp[\![e_1]\!](\sigma^\sharp), [c, d] = \mathbb{E}^\sharp[\![e_2]\!](\sigma^\sharp)$.

$$\begin{aligned}
\mathbb{E}^\sharp[\![e_1 + e_2]\!](\sigma^\sharp) &= \alpha(\mathbb{E}'[\![e_1 + e_2]\!](\gamma(\sigma^\sharp))) \\
&\sqsubseteq \alpha(\{x + y \mid x \in [a, b], y \in [c, d]\}) \\
&= [\min(\{x + y \mid x \in [a, b], y \in [c, d]\}), \max(\cdots)] \\
&= [a + c, b + d]
\end{aligned}$$

wow it's just interval arithmetic!

Remaining interval semantics

We still have to write out $\mathbb{C}^\sharp[\![c]\!]$ and $\mathbb{S}^\sharp[\![s]\!]$ ourselves.

Remaining interval semantics

We still have to write out $\mathbb{C}^\sharp[\![c]\!]$ and $\mathbb{S}^\sharp[\![s]\!]$ ourselves.

$$\begin{aligned}
\mathbb{C}^\sharp[\![v < n]\!](\sigma^\sharp) &= \sigma[v \mapsto \sigma(v) \sqcap [-\infty, n-1]] \\
\mathbb{C}^\sharp[\![v = n]\!](\sigma^\sharp) &= \sigma[v \mapsto \sigma(v) \sqcap [n, n]] \text{ etc} \\
\mathbb{C}^\sharp[\![c]\!](\sigma^\sharp) &= \sigma^\sharp
\end{aligned}$$

Remaining interval semantics

We still have to write out $\mathbb{C}^\sharp[\![c]\!]$ and $\mathbb{S}^\sharp[\![s]\!]$ ourselves.

$$\begin{aligned}
\mathbb{C}^\sharp[\![v < n]\!](\sigma^\sharp) &= \sigma[v \mapsto \sigma(v) \sqcap [-\infty, n-1]] \\
\mathbb{C}^\sharp[\![v = n]\!](\sigma^\sharp) &= \sigma[v \mapsto \sigma(v) \sqcap [n, n]] \text{ etc} \\
\mathbb{C}^\sharp[\![c]\!](\sigma^\sharp) &= \sigma^\sharp
\end{aligned}$$

$$\mathbb{S}^\sharp[\![v \leftarrow e]\!](S) = \sigma[v \mapsto \mathbb{E}^\sharp[\![e]\!](\sigma)]$$
$$\mathbb{S}^\sharp[\![s_1; s_2]\!](S) = \mathbb{S}^\sharp[\![s_2]\!](\mathbb{S}^\sharp[\![s_1]\!](S))$$
$$\mathbb{S}^\sharp[\![\textbf{if } c \textbf{ then } s_1 \textbf{ else } s_2 \textbf{ end if}]\!](S) = \mathbb{S}^\sharp[\![s_1]\!](\mathbb{C}^\sharp[\![c]\!](S)) \cup \mathbb{S}^\sharp[\![s_1]\!](\mathbb{C}^\sharp[\![\neg c]\!](S))$$
$$\mathbb{S}^\sharp[\![\textbf{while } c \textbf{ do } s \textbf{ end while}]\!](S) = \mathbb{C}^\sharp[\![\neg c]\!](\mathrm{lfp}(X \mapsto X \cup \mathbb{S}^\sharp[\![s]\!](\mathbb{C}^\sharp[\![c]\!](X))))$$

Remaining interval semantics

We still have to write out $\mathbb{C}^\sharp[\![c]\!]$ and $\mathbb{S}^\sharp[\![s]\!]$ ourselves.

$$\begin{aligned}
\mathbb{C}^\sharp[\![v < n]\!](\sigma^\sharp) &= \sigma[v \mapsto \sigma(v) \sqcap [-\infty, n-1]] \\
\mathbb{C}^\sharp[\![v = n]\!](\sigma^\sharp) &= \sigma[v \mapsto \sigma(v) \sqcap [n, n]] \text{ etc} \\
\mathbb{C}^\sharp[\![c]\!](\sigma^\sharp) &= \sigma^\sharp
\end{aligned}$$

$$\mathbb{S}^\sharp[\![v \leftarrow e]\!](S) = \sigma[v \mapsto \mathbb{E}^\sharp[\![e]\!](\sigma)]$$
$$\mathbb{S}^\sharp[\![s_1; s_2]\!](S) = \mathbb{S}^\sharp[\![s_2]\!](\mathbb{S}^\sharp[\![s_1]\!](S))$$
$$\mathbb{S}^\sharp[\![\textbf{if } c \textbf{ then } s_1 \textbf{ else } s_2 \textbf{ end if}]\!](S) = \mathbb{S}^\sharp[\![s_1]\!](\mathbb{C}^\sharp[\![c]\!](S)) \cup \mathbb{S}^\sharp[\![s_1]\!](\mathbb{C}^\sharp[\![\neg c]\!](S))$$
$$\mathbb{S}^\sharp[\![\textbf{while } c \textbf{ do } s \textbf{ end while}]\!](S) = \mathbb{C}^\sharp[\![\neg c]\!](\mathrm{lfp}(X \mapsto X \cup \mathbb{S}^\sharp[\![s]\!](\mathbb{C}^\sharp[\![c]\!](X))))$$

Wow, this is almost all computable now!! The only thing that isn't is the while loop... It's a join of infinitely many things!

Fixpoint acceleration

If we have a sequence of intervals $([0,0],[0,1],[0,2],[0,3],\dots)$, then there's no hope to find a fixed-point of them by joining them repeatedly.

Fixpoint acceleration

If we have a sequence of intervals $([0,0],[0,1],[0,2],[0,3],\dots)$, then there's no hope to find a fixed-point of them by joining them repeatedly.
We can widen intervals like so:

$$[a,b] \nabla [c,d] = \left[\begin{cases} a & \text{if } a \leq c \\ -\infty & \text{otherwise} \end{cases}, \begin{cases} b & \text{if } b \geq d \\ \infty & \text{otherwise} \end{cases} \right].$$

Intuitively, if we see $[c,d]$ is growing, clamp to infinity.
This is an instance of what is called a widening operator (definition omitted)!

Fixpoint acceleration

If we have a sequence of intervals $([0,0],[0,1],[0,2],[0,3],\dots)$, then there's no hope to find a fixed-point of them by joining them repeatedly.
We can widen intervals like so:

$$[a,b] \nabla [c,d] = \left[\begin{cases} a & \text{if } a \leq c \\ -\infty & \text{otherwise} \end{cases}, \begin{cases} b & \text{if } b \geq d \\ \infty & \text{otherwise} \end{cases} \right].$$

Intuitively, if we see $[c,d]$ is growing, clamp to infinity.
This is an instance of what is called a widening operator (definition omitted)!

Theorem

If $g : A \to A$ is an approximation of $f : C \to C$ (in the sense of the (lax) commutative diagram from earlier), then the sequence $x_0 = \bot$ and $x_{i+1} = x_i \nabla g(x_i)$ stabilises in finite time to x to approximate the least fixed point of f, $\mathrm{lfp}(f) \sqsubseteq \gamma(x)$.

no proof

Fixpoint acceleration

Fixpoint acceleration

We can then define widening on I^V pointwise, and define while semantics with

$$\mathbb{S}^\sharp[\![\textbf{while } c \textbf{ do } s \textbf{ end while}]\!](S) = \\ \mathbb{C}^\sharp[\![\neg c]\!](\mathsf{lfp}(X \mapsto X \nabla (S \sqcup \mathbb{S}^\sharp[\![s]\!](\mathbb{C}^\sharp[\![c]\!](X))))).$$

This is computable!

Fixpoint acceleration

We can then define widening on I^V pointwise, and define while semantics with

$$\mathbb{S}^\sharp[\![\textbf{while } c \textbf{ do } s \textbf{ end while}]\!](S) = \mathbb{C}^\sharp[\![\neg c]\!](\mathrm{lfp}(X \mapsto X \nabla (S \sqcup \mathbb{S}^\sharp[\![s]\!](\mathbb{C}^\sharp[\![c]\!](X))))).$$

This is computable!
Also everything here is an approximation of the concrete semantics i promise

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$

$F[0] \leftarrow 0$

$F[1] \leftarrow 1$

$i \leftarrow 2$

while $i < 100$ **do**

  if $2 > i \vee i > 99$ **then**

    explode (out of bounds)

  end if

  $F[i] \leftarrow F[i-1] + F[i-2]$

  $i \leftarrow i + 1$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**

if $2 > i \vee i > 99$ **then**
explode (out of bounds)
end if
$F[i] \leftarrow F[i-1] + F[i-2]$
$i \leftarrow i + 1$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$
while $i < 100$ **do**

  if $2 > i \vee i > 99$ **then**
    explode (out of bounds)
  end if
  $F[i] \leftarrow F[i-1] + F[i-2]$
  $i \leftarrow i + 1$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**

  if $2 > i \vee i > 99$ **then**
    explode (out of bounds)
  end if
  $F[i] \leftarrow F[i-1] + F[i-2]$
  $i \leftarrow i + 1$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 2] \sqcap [-\infty, 99]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$
    $i \leftarrow i + 1$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 2]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$
    $i \leftarrow i + 1$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 2]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 2]$
    $i \leftarrow i + 1$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 2]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 2]$
    $i \leftarrow i + 1$ $i \mapsto [3, 3]$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$

$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$

$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$

$i \leftarrow 2$ $i \mapsto [2, 2]$

while $i < 100$ **do**

  $i \mapsto [2, 2]$

  if $2 > i \vee i > 99$ **then**

    explode (out of bounds)

  end if

  $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 2]$

  $i \leftarrow i + 1$ $i \mapsto [3, 3]$

  $i \mapsto [2, 2] \nabla ([2, 2] \sqcup [3, 3])$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 2]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 2]$
    $i \leftarrow i + 1$ $i \mapsto [3, 3]$
    $i \mapsto [2, 2] \nabla [2, 3]$
end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 2]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 2]$
    $i \leftarrow i + 1$ $i \mapsto [3, 3]$
    $i \mapsto [2, \infty]$
end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, \infty] \sqcap [-\infty, 99]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 2]$
    $i \leftarrow i + 1$ $i \mapsto [3, 3]$
    $i \mapsto [2, \infty]$
end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 99]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 2]$
    $i \leftarrow i + 1$ $i \mapsto [3, 3]$
    $i \mapsto [2, \infty]$
end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 99]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 99]$
    $i \leftarrow i + 1$ $i \mapsto [3, 3]$
    $i \mapsto [2, \infty]$
end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 99]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 99]$
    $i \leftarrow i + 1$ $i \mapsto [3, 100]$
    $i \mapsto [2, \infty]$
end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$

$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$

$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$

$i \leftarrow 2$ $i \mapsto [2, 2]$

while $i < 100$ **do**

$i \mapsto [2, 99]$

if $2 > i \vee i > 99$ **then**

explode (out of bounds)

end if

$F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 99]$

$i \leftarrow i + 1$ $i \mapsto [3, 100]$

$i \mapsto [2, 99] \nabla ([2, 2] \sqcup [3, 100])$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$

$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$

$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$

$i \leftarrow 2$ $i \mapsto [2, 2]$

while $i < 100$ **do**

$i \mapsto [2, 99]$

if $2 > i \vee i > 99$ **then**

explode (out of bounds)

end if

$F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 99]$

$i \leftarrow i + 1$ $i \mapsto [3, 100]$

$i \mapsto [2, 99] \bigtriangledown [2, 100]$

end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 99]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 99]$
    $i \leftarrow i + 1$ $i \mapsto [3, 100]$
    $i \mapsto [2, \infty]$
end while

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$
$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$
$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$
$i \leftarrow 2$ $i \mapsto [2, 2]$
while $i < 100$ **do**
    $i \mapsto [2, 99]$
    if $2 > i \vee i > 99$ **then**
        explode (out of bounds)
    end if
    $F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 99]$
    $i \leftarrow i + 1$ $i \mapsto [3, 100]$
    $i \mapsto [2, \infty]$
end while
$i \mapsto [100, \infty]$
return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$ $i \mapsto [\infty, \infty]$

$F[0] \leftarrow 0$ $i \mapsto [\infty, \infty]$

$F[1] \leftarrow 1$ $i \mapsto [\infty, \infty]$

$i \leftarrow 2$ $i \mapsto [2, 2]$

while $i < 100$ **do**

$i \mapsto [2, 99]$

$F[i] \leftarrow F[i-1] + F[i-2]$ $i \mapsto [2, 99]$

$i \leftarrow i + 1$ $i \mapsto [3, 100]$

$i \mapsto [2, \infty]$

end while

$i \mapsto [100, \infty]$

return F

Abstract interpreting

$F \leftarrow \underbrace{[0, \ldots, 0]}_{\text{100 of them}}$
$F[0] \leftarrow 0$
$F[1] \leftarrow 1$
$i \leftarrow 2$
while $i < 100$ **do**
    $F[i] \leftarrow F[i-1] + F[i-2]$
    $i \leftarrow i + 1$
end while
return F

we did it!!

Further reading



Patrick Cousot and Radhia Cousot.
Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints.
In Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, POPL '77, page 238–252, New York, NY, USA, 1977. Association for Computing Machinery.



Antoine Miné.
Tutorial on static inference of numeric invariants by abstract interpretation.
Found. Trends Program. Lang., 4(3–4):120–372, December 2017.